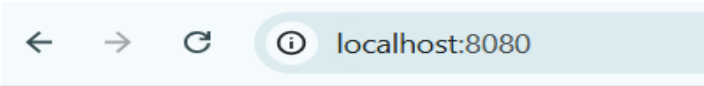
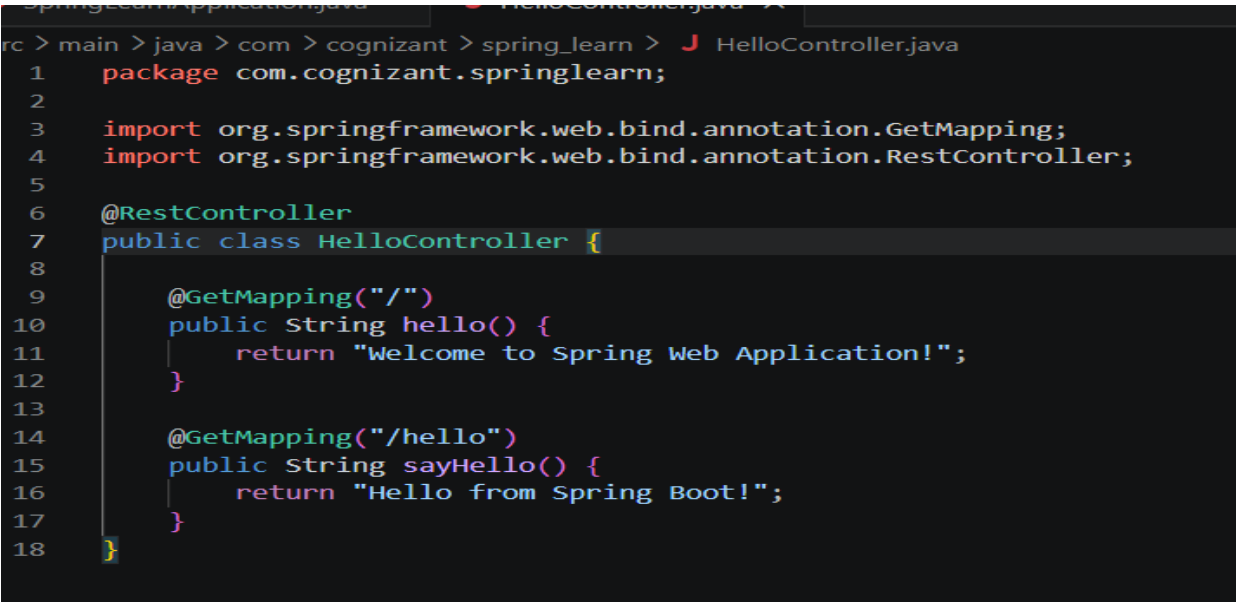
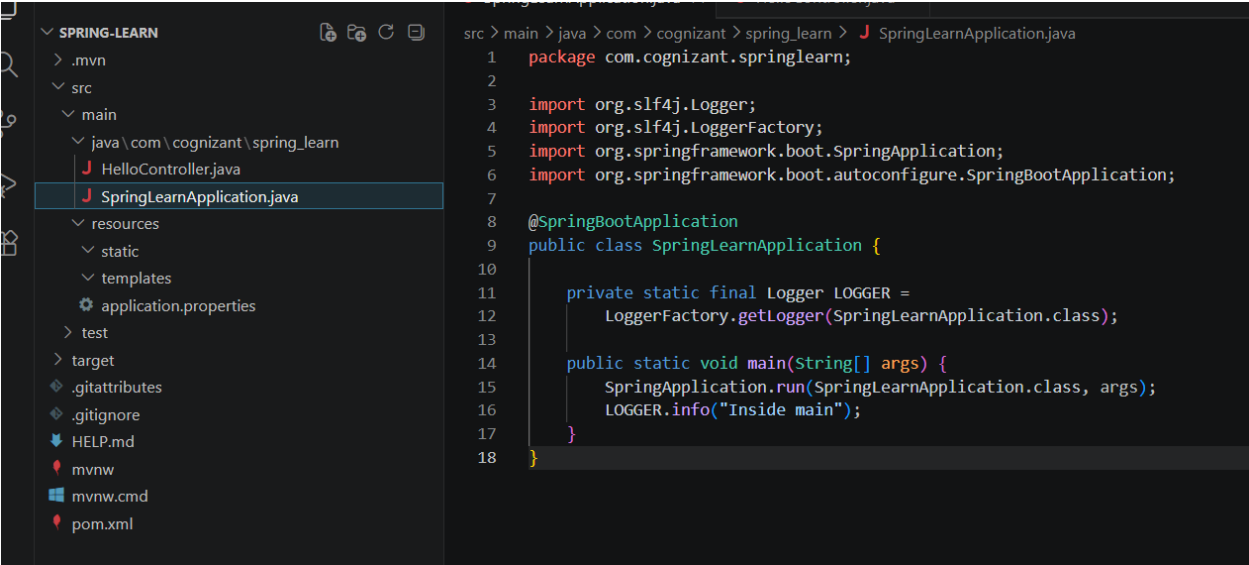
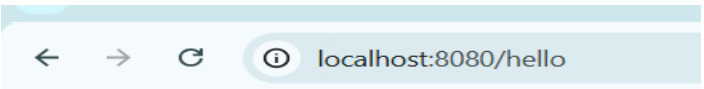


spring-rest

1.Create a Spring Web Project using Maven



Welcome to Spring Web Application!



Hello from Spring Boot!

2. Spring Core – Load Country from Spring Configuration XML



```
main / java / com / cognizant / springlearn / HelloController.java
package com.cognizant.springlearn;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/")
    public String hello() {
        return "Welcome to Spring Web Application!";
    }

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello from Spring Boot!";
    }
}
```

```
main / java / com / cognizant / springlearn / SpringLearnApplication.java
package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
        LOGGER.info("Inside main");
        displayCountry();
    }

    private static void displayCountry() {
        LOGGER.info("Start");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = (Country) context.getBean("country", Country.class);
        LOGGER.debug("Country : {}", country.toString());
        LOGGER.info("End");
    }
}
```

```
alization completed in 473 ms
2026-07-01T14:25:21.201+05:30 INFO 8608 --- [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on p
ort 35729
2026-07-01T14:25:21.217+05:30 INFO 8608 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8082 (http
) with context path '/'
2026-07-01T14:25:21.222+05:30 INFO 8608 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in
0.937 seconds (process running for 1.137)
2026-07-01T14:25:21.223+05:30 INFO 8608 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : Inside main
2026-07-01T14:25:21.224+05:30 INFO 8608 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : Start
2026-07-01T14:25:21.275+05:30 DEBUG 8608 --- [ restartedMain] com.cognizant.springlearn.Country : Inside Country Constructor.
2026-07-01T14:25:21.276+05:30 DEBUG 8608 --- [ restartedMain] com.cognizant.springlearn.Country : Inside setCode. code=IN
2026-07-01T14:25:21.277+05:30 DEBUG 8608 --- [ restartedMain] com.cognizant.springlearn.Country : Inside setName. name=India
2026-07-01T14:25:21.278+05:30 DEBUG 8608 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : Country : Country [code=IN, name=
India]
2026-07-01T14:25:21.278+05:30 INFO 8608 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : End
[]
```

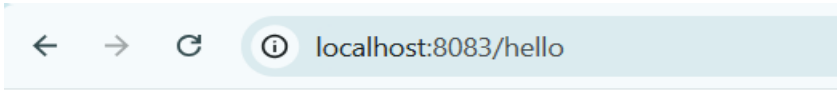
3. Hello World RESTful Web Service

```
src > main > resources > application.properties
1 server.port=8083
2 logging.level.com.cognizant=debug
```

```
src > main / java / com / cognizant / springlearn / HelloController.java
1 package com.cognizant.springlearn.controller;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.web.bind.annotation.GetMapping;
6 import org.springframework.web.bind.annotation.RestController;
7
8 @RestController
9 public class HelloController {
10
11     private static final Logger LOGGER =
12         LoggerFactory.getLogger(HelloController.class);
13
14     @GetMapping("/hello")
15     public String sayHello() {
16         LOGGER.info("Start");
17         LOGGER.info("End");
18         return "Hello World!!";
19     }
20 }
```

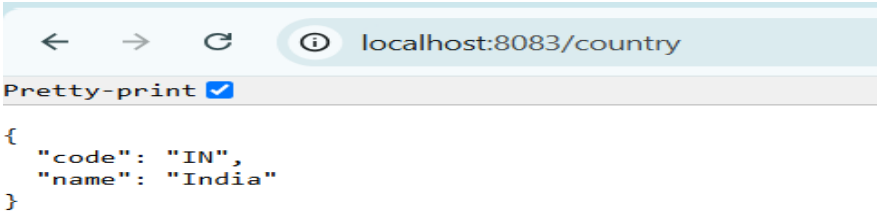
4.REST - Country Web Service

```
> main > resources > country.xml
1  <?xml version="1.0" encoding="UTF-8"?>
2  <beans xmlns="http://www.springframework.org/schema/beans"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://www.springframework.org/schema/beans
5          http://www.springframework.org/schema/beans/spring-beans.xsd">
6
7      <bean id="country" class="com.cognizant.springlearn.Country">
8          <property name="code" value="IN" />
9          <property name="name" value="India" />
10     </bean>
11
12 </beans>
13
14
15
```



Hello World!!

```
src > main > java > com > cognizant > springlearn > controller > CountryController.java
7  import org.springframework.web.bind.annotation.RequestMapping;
8  import org.springframework.web.bind.annotation.RestController;
9  import com.cognizant.springlearn.Country;
10
11  @RestController
12  public class CountryController {
13
14      private static final Logger LOGGER =
15          LoggerFactory.getLogger(CountryController.class);
16
17      @RequestMapping("/country")
18      public Country getCountryIndia() {
19          LOGGER.info("Start");
20          ApplicationContext context =
21              new ClassPathXmlApplicationContext("country.xml");
22          Country country = context.getBean("country", Country.class);
23          LOGGER.info("End");
24          return country;
25      }
26  }
```



5.REST - Get country based on country code

```
package com.cognizant.springlearn.service;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;
import com.cognizant.springlearn.Country;

@Service
public class CountryService {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryService.class);

    public Country getCountry(String code) {
        LOGGER.info("Start");
        LOGGER.debug("code={}", code);

        // Step 1: Load country list from country.xml
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");

        // Step 2: Get the list of all countries
        List<Country> countryList =
            (List<Country>) context.getBean("countryList");

        // Step 3: Find country using Lambda - case insensitive match
        Country country = countryList.stream()
            .filter(c -> c.getCode().equalsIgnoreCase(code))
            .findFirst()
            .orElse(null);

        LOGGER.debug("country={}", country);
        LOGGER.info("End");
        return country;
    }
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="countryList" class="java.util.ArrayList">
        <constructor-arg>
            <list>
                <bean class="com.cognizant.springlearn.Country">
                    <property name="code" value="IN" />
                    <property name="name" value="India" />
                </bean>
                <bean class="com.cognizant.springlearn.Country">
                    <property name="code" value="US" />
                    <property name="name" value="United States" />
                </bean>
                <bean class="com.cognizant.springlearn.Country">
                    <property name="code" value="DE" />
                    <property name="name" value="Germany" />
                </bean>
                <bean class="com.cognizant.springlearn.Country">
                    <property name="code" value="JP" />
                    <property name="name" value="Japan" />
                </bean>
            </list>
        </constructor-arg>
    </bean>

    <bean id="country" class="com.cognizant.springlearn.Country">
        <property name="code" value="IN" />
        <property name="name" value="India" />
    </bean>

</beans>
```

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;

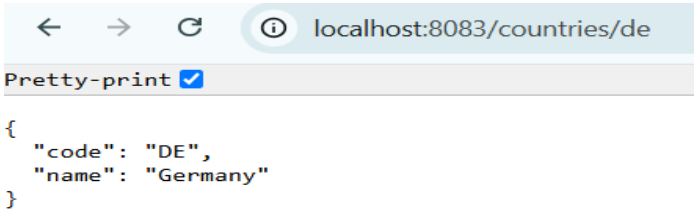
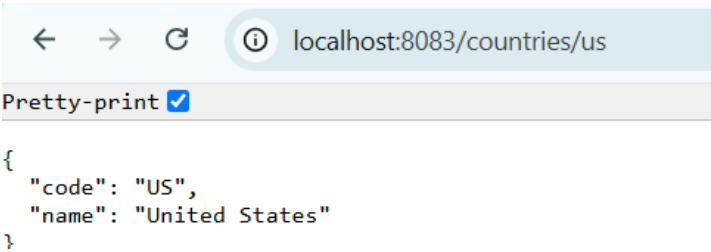
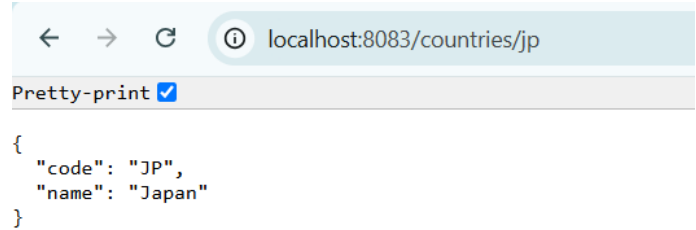
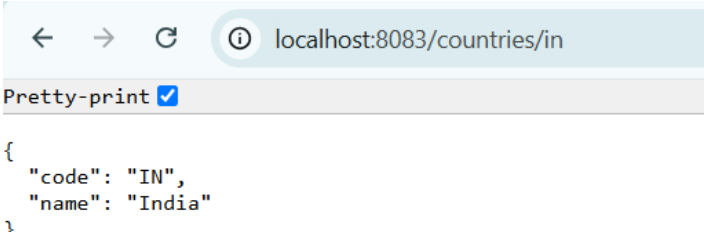
@RestController
public class CountryController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);

    @Autowired
    private CountryService countryService;

    // Existing endpoint
    @RequestMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("Start");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.info("End");
        return country;
    }

    // New endpoint - get country by code
    @GetMapping("/countries/{code}")
    public Country getCountry(@PathVariable String code) {
        LOGGER.info("Start");
        LOGGER.debug("code={}", code);
        Country country = countryService.getCountry(code);
        LOGGER.info("End");
        return country;
    }
}
```



6. Create authentication service that returns JWT

```
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
7 import org.springframework.security.web.SecurityFilterChain;
8
9 @Configuration
10 @EnableWebSecurity
11 public class SecurityConfig {
12
13     @Bean
14     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
15
16         http
17             .csrf(csrf -> csrf.disable())
18             .authorizeHttpRequests(auth -> auth
19                 // Allow /authenticate without login
20                 .requestMatchers("/authenticate").permitAll()
21                 // All other URLs need authentication
22                 .anyRequest().authenticated()
23             )
24             // Use HTTP Basic Authentication
25             .httpBasic(basic -> basic.realmName("spring-learn"));
26
27         return http.build();
28     }
29 }
```

```
package com.cognizant.springlearn.controller;

import com.cognizant.springlearn.model.AuthResponse;
import com.cognizant.springlearn.util.JwtUtil;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpHeaders;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;
import java.util.Base64;

@RestController
public class AuthenticationController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(AuthenticationController.class);

    @Autowired
    private JwtUtil jwtUtil;

    @GetMapping("/authenticate")
    public AuthResponse authenticate(
        @RequestHeader(HttpHeaders.AUTHORIZATION) String authHeader) {

        LOGGER.info("Start");

        // Step 1: Read Authorization header
        // Header format: "Basic dXNlcjpwd2Q="
        LOGGER.debug("authHeader={}", authHeader);

        // Step 2: Decode Base64 credentials
        // Remove "Basic " prefix and decode
        String base64Credentials = authHeader.substring(6);
        String decoded = new String(Base64.getDecoder().decode(base64Credentials));

        // decoded = "user:pwd"
        String[] credentials = decoded.split(":");
        String username = credentials[0];
        String password = credentials[1];

        LOGGER.debug("username={}", username);

        // Step 3: Generate JWT token
        String token = jwtUtil.generateToken(username);
        LOGGER.debug("token={}", token);
    }
}
```

```
src / main / java / com / cognizant / springlearn / model / AuthRequest.java
1 package com.cognizant.springlearn.model;
2
3 public class AuthRequest {
4
5     private String username;
6     private String password;
7
8     public String getUsername() { return username; }
9     public void setUsername(String username) { this.username = username; }
10    public String getPassword() { return password; }
11    public void setPassword(String password) { this.password = password; }
12 }
```

```
src > main > java > com > cognizant > springlearn > model > J AuthResponse.java
1 package com.cognizant.springlearn.model;
2
3 public class AuthResponse {
4
5     private String token;
6
7     public AuthResponse(String token) {
8         this.token = token;
9     }
10
11     public String getToken() { return token; }
12     public void setToken(String token) { this.token = token; }
13 }
```

```
> main > java > com > cognizant > springlearn > util > J JwtUtil.java
1 package com.cognizant.springlearn.util;
2
3 import io.jsonwebtoken.Jwts;
4 import io.jsonwebtoken.SignatureAlgorithm;
5 import io.jsonwebtoken.security.Keys;
6 import org.springframework.stereotype.Component;
7 import java.security.Key;
8 import java.util.Date;
9
10 @Component
11 public class JwtUtil {
12
13     // Secret key for signing JWT
14     private final Key key = Keys.secretKeyFor(SignatureAlgorithm.HS256);
15
16     // Token valid for 20 minutes
17     private final long EXPIRATION = 1000 * 60 * 20;
18
19     public String generateToken(String username) {
20         return Jwts.builder()
21             .setSubject(username)
22             .setIssuedAt(new Date())
23             .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION))
24             .signWith(key)
25             .compact();
26     }
27
28     public String extractUsername(String token) {
29         return Jwts.parserBuilder()
30             .setSigningKey(key)
31             .build()
32             .parseClaimsJws(token)
33             .getBody()
34             .getSubject();
35     }
36
37     public boolean validateToken(String token) {
38         try {
39             Jwts.parserBuilder()
40                 .setSigningKey(key)
41                 .build()
42                 .parseClaimsJws(token);
43             return true;
44         } catch (Exception e) {
45             return false;
46         }
47     }
48 }
```

localhost:8090/authenticate

Sign in to access this site

Authorization required by http://localhost:8090

Username

Password

Sign in

Cancel

```
< > localhost:8090/authenticate
Pretty-print
{
  "token": "eyJhbGciOiJIUzI1NiIsInp0eSI6dWUiOiJlc2Vybm9iaWF0IjoxNzgyOTc1NjM2LCJleHAiOjE3ODI5NzY4MzZ9.5SycyVi310CWnndVES1wJWVbQ6t0cMk0kuuQyZLbc0k"
}
```